Neural Dependency Parsing

CS 115B – Spring 2026

James Pustejovsky

February 26, 2026

Roadmap for Today

Part I: Dependency Parsing Theory (20 minutes)

- What are dependencies?
- Dependency grammar vs. phrase structure
- Dependency trees and properties

Part II: Transition-Based Parsing (25 minutes)

- Arc-standard transition system
- Stack, buffer, and actions
- Complete parsing examples

Part III: Chen & Manning Neural Parser (35 minutes)

- Motivation: Feature engineering problem
- Neural network architecture
- Training and inference
- Detailed examples

Part IV: Results and Impact (10 minutes)

What is Syntactic Structure?

Goal: Represent grammatical relationships between words

Two main formalisms:

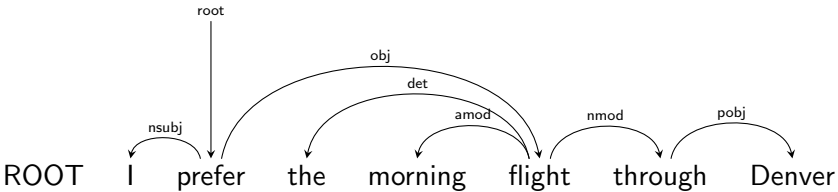
1. **Phrase structure (constituency)**:
   - Groups words into nested constituents
   - Example: (S (NP The cat) (VP (V sat) (PP (P on) (NP the mat))))
   - Focus on hierarchical groupings

2. **Dependency structure**:
   - Direct binary relations between words
   - Example: "sat" depends on ROOT, "cat" depends on "sat"
   - Focus on head-dependent relations

Today: Dependency parsing!

Dependency Structure: Example

Sentence: "I prefer the morning flight through Denver"



Dependencies:

- "prefer" is the **head** of "I" (subject relation)
- "prefer" is the head of "flight" (object relation)
- "flight" is the head of "the", "morning" (modifiers)
- "through" is the head of "Denver" (prepositional object)

Dependency Relations

Each arc has a label indicating the grammatical relation:

Common Universal Dependencies relations:

- **nsubj**: Nominal subject ("I" $\leftarrow$ "prefer")
- **obj**: Direct object ("flight" $\leftarrow$ "prefer")
- **det**: Determiner ("the" $\leftarrow$ "flight")
- **amod**: Adjectival modifier ("morning" $\leftarrow$ "flight")
- **nmod**: Nominal modifier ("through" $\leftarrow$ "flight")
- **case**: Case marker ("Denver" $\leftarrow$ "through")
- **root**: Relation to ROOT

Universal Dependencies (UD) v2 has 40 core relation types

Note: Chen & Manning focus on unlabeled parsing first, then add labels

Why Dependencies?

Advantages over constituency parsing:

1. **Word-to-word relations**:
   - Direct connection between "prefer" and "flight"
   - No need for intermediate nodes (VP, NP, etc.)
   - Easier for downstream tasks (relation extraction, QA)
2. **Better for free word order languages**:
   - Czech, Russian, Latin: word order flexible
   - Dependencies capture relations regardless of order
3. **Useful for NLP applications**:
   - Information extraction: who did what to whom?
   - Machine translation: align dependencies across languages
   - Question answering: find semantic roles

Dependency Tree Properties

Valid dependency tree must satisfy:

1. **Single head**: Each word has exactly one head (except ROOT)
2. **Acyclic**: No cycles (cannot have $a \rightarrow b \rightarrow c \rightarrow a$)
3. **Connected**: All words connected to tree
4. **Rooted**: One unique root node

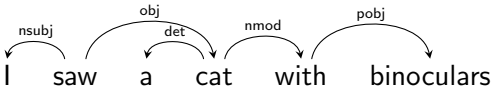
These properties define a **spanning tree** over the words

Projectivity (optional constraint):

- Tree is projective if no crossing arcs
- Equivalent to: can draw all arcs above words without crossing
- Most English sentences are projective
- Some languages (Czech, Dutch) have more non-projective trees

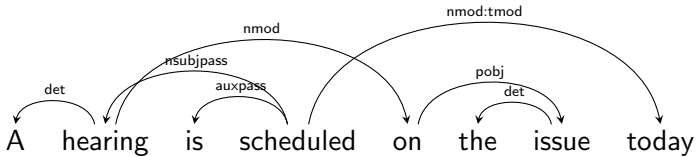
Projective vs. Non-Projective

Projective (common in English):



No crossing arcs

Non-projective (rare in English, common in Czech):



"hearing" linked to "on" crosses over with "scheduled" linked to "today" (non-projective arc)

Dependency Parsing Approaches

Three main algorithmic families:

1. **Graph-based (Eisner, MST)**:
   - Score all possible arcs
   - Find maximum spanning tree
   - Global optimization, can be non-projective
   - $O(n^3)$ for projective, $O(n^2)$ for approximate non-projective
2. **Transition-based (shift-reduce)**:
   - Process words left-to-right
   - Build tree incrementally with actions
   - Greedy or beam search, linear time
   - ← Chen & Manning use this!
3. **Hybrid / Neural graph-based**:
   - Modern: Dozat & Manning (2017) biaffine parser
   - Encode with BERT, score all arcs with neural network

Transition-Based Parsing: Overview

Key idea: Build parse tree through sequence of actions

Configuration:

- **Stack** σ: Partially processed words
- **Buffer** β: Remaining words to process
- **Arcs** A: Dependencies built so far

Actions (arc-standard system):

- **SHIFT**: Move first word from buffer to stack
- **LEFT-ARC**: Add arc from stack top to second-to-top, remove second-to-top
- **RIGHT-ARC**: Add arc from second-to-top to stack top, remove top

Goal: Start with all words in buffer, end with all arcs created

Arc-Standard Actions: Details

Initial configuration:

- Stack: [ROOT]
- Buffer: [word$_1$, word$_2$, ..., word$_n$]
- Arcs: $\emptyset$

Action definitions:

SHIFT: $(\sigma, w_i|\beta, A) \Rightarrow (\sigma|w_i, \beta, A)$

- Move word from buffer to stack

LEFT-ARC$_r$: $(\sigma|w_i|w_j, \beta, A) \Rightarrow (\sigma|w_j, \beta, A \cup \{w_j \xrightarrow{r} w_i\})$

- Add arc w_j (stack top) $\rightarrow$ w_i (second) with label r
- Remove w_i from stack

RIGHT-ARC$_r$: $(\sigma|w_i|w_j, \beta, A) \Rightarrow (\sigma|w_i, \beta, A \cup \{w_i \xrightarrow{r} w_j\})$

- Add arc w_i (second) $\rightarrow$ w_j (top) with label r
- Remove w_j from stack

Transition Example: "I eat fish"

Sentence: "I eat fish"
Target dependencies:

- ROOT $\rightarrow$ eat (root)
- eat $\rightarrow$ I (nsubj)
- eat $\rightarrow$ fish (obj)

Initial state:

- Stack: [ROOT]
- Buffer: [I, eat, fish]
- Arcs: $\emptyset$

We'll trace through the parsing actions step by step...

Step 1: SHIFT

Action: SHIFT

Before:

- Stack: [ROOT]
- Buffer: [I, eat, fish]
- Arcs: $\emptyset$

After:

- Stack: [ROOT, I]
- Buffer: [eat, fish]
- Arcs: $\emptyset$

Move "I" from buffer to stack to process it

Step 2: SHIFT

Action: SHIFT

Before:

- Stack: [ROOT, I]
- Buffer: [eat, fish]
- Arcs: $\emptyset$

After:

- Stack: [ROOT, I, eat]
- Buffer: [fish]
- Arcs: $\emptyset$

Move "eat" to stack. Now we have "I" and "eat" on stack, can create arc

Step 3: LEFT-ARC (nsubj)

Action: LEFT-ARC$_{\text{nsubj}}$

Before:

- Stack: [ROOT, I, eat]
- Buffer: [fish]
- Arcs: $\emptyset$

After:

- Stack: [ROOT, eat]
- Buffer: [fish]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}\}$

Create dependency: "eat" (top) is the head of "I" (second). Remove "I" from stack since it's complete.

Notation: Stack top, Second from top

Step 4: SHIFT

Action: SHIFT

Before:

- Stack: [ROOT, eat]
- Buffer: [fish]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}\}$

After:

- Stack: [ROOT, eat, fish]
- Buffer: [ ]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}\}$

Move "fish" to stack

Step 5: RIGHT-ARC (obj)

Action: RIGHT-ARC$_{\text{obj}}$

Before:

- Stack: [ROOT, eat, fish]
- Buffer: [ ]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}\}$

After:

- Stack: [ROOT, eat]
- Buffer: [ ]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}, \text{eat} \xrightarrow{\text{obj}} \text{fish}\}$

Create dependency: "eat" (second) is head of "fish" (top). Remove "fish".

Step 6: RIGHT-ARC (root)

Action: $\text{RIGHT-ARC}_{\text{root}}$

Before:

- Stack: [ROOT, eat]
- Buffer: [ ]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}, \text{eat} \xrightarrow{\text{obj}} \text{fish}\}$

After:

- Stack: [ROOT]
- Buffer: [ ]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}, \text{eat} \xrightarrow{\text{obj}} \text{fish}, \text{ROOT} \xrightarrow{\text{root}} \text{eat}\}$

Parsing complete! Stack has only ROOT, buffer empty, all arcs created.

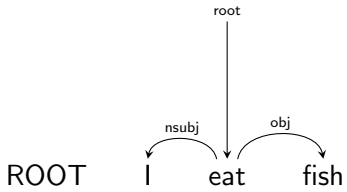
Complete Action Sequence

Sentence: "I eat fish"

Step	Action	Stack	Buffer
0	—	[ROOT]	[I, eat, fish]
1	SHIFT	[ROOT, I]	[eat, fish]
2	SHIFT	[ROOT, I, eat]	[fish]
3	LEFT-ARC$_{\text{nsubj}}$	[ROOT, eat]	[fish]
4	SHIFT	[ROOT, eat, fish]	[ ]
5	RIGHT-ARC$_{\text{obj}}$	[ROOT, eat]	[ ]
6	RIGHT-ARC$_{\text{root}}$	[ROOT]	[ ]

Complete Resulting Tree

Sentence: "I eat fish"



Action Preconditions

Question: Can we always apply any action?

Answer: No! Actions have preconditions:

SHIFT:

- Precondition: Buffer must be non-empty

LEFT-ARC:

- Precondition: Stack must have ≥ 2 elements
- Additional: Second element cannot be ROOT (ROOT has no head)

RIGHT-ARC:

- Precondition: Stack must have ≥ 2 elements
- Additional: Top element must not already have a head (single head constraint)

Oracle: Supervision signal that tells us the correct action given gold tree

Greedy vs. Beam Search

Greedy parsing:

- At each step, pick highest-scoring valid action
- Fast: $O(n)$ actions for sentence of length n
- Problem: Early mistakes cannot be corrected

Beam search:

- Maintain top-k partial parses
- At each step, expand all and keep best k
- Better accuracy, slower: $O(nkA)$ where A is action count
- Chen & Manning use greedy (later work adds beam search)

Trade-off: Speed vs. accuracy

- Greedy: 1000 sentences/second
- Beam search (k=8): 100 sentences/second

Traditional Approach: Feature Templates

Pre-neural parsers (Nivre, Zhang & Nivre 2011):

Feature extraction from configuration:

- Top 3 words on stack: s_1, s_2, s_3
- First 3 words in buffer: b_1, b_2, b_3
- Leftmost/rightmost children of stack words
- POS tags of these words
- Dependency labels of existing arcs

Example features:

- Word form: $s_1.w$, $b_1.w$, $s_2.w$
- POS tag: $s_1.t$, $b_1.t$
- Combinations: $\langle s_1.w, s_1.t \rangle$, $\langle s_1.w, b_1.w \rangle$
- Contextual: $\langle s_1.w, s_1.lc.w \rangle$ (leftmost child)

Typical feature set: 10-20 million sparse binary features!

Problems with Feature Templates

1. **Incompleteness**:
   - Hand-designed features miss important patterns
   - Cannot easily add higher-order features (combinations explode)
   - Expert knowledge required
2. **Sparsity**:
   - Most features are 0 in any configuration
   - Need millions of training examples to learn weights
   - Rare word combinations never seen
3. **Computational cost**:
   - Feature extraction is expensive (lookup many templates)
   - 95% of parsing time spent on feature computation!
   - Each configuration: extract features $\rightarrow$ score actions

Chen & Manning's insight: Use neural network to learn features!

Key Innovation: Learned Feature Embeddings

Instead of: Millions of sparse binary features

Use: Dense continuous embeddings

Representation:

- Words: d-dimensional vectors (e.g., 50-dim)
- POS tags: d-dimensional vectors
- Dependency labels: d-dimensional vectors

Advantages:

- Automatic feature learning
- Generalization: similar words have similar embeddings
- Dense representation: every word has informative vector
- Compositional: can combine features easily

Input Representation

From configuration, extract:

Stack elements (Chen & Manning use top 3):

- s_1, s_2, s_3: Top three words on stack

Buffer elements (first 3):

- b_1, b_2, b_3: First three words in buffer

Children (leftmost/rightmost 2 children of s_1, s_2):

- $lc_1(s_1), rc_1(s_1), lc_2(s_1), rc_2(s_1)$
- $lc_1(s_2), rc_1(s_2), lc_2(s_2), rc_2(s_2)$

Total: 18 elements (positions to extract information from)

For each element, get:

- Word embedding
- POS tag embedding
- (For children) Dependency label embedding

Feature Set Size

Token elements: 18 positions

For each position:

- Word: $w \in \mathbb{R}^{d_w}$ (word embedding)
- POS tag: $t \in \mathbb{R}^{d_t}$ (tag embedding)
- Label: $l \in \mathbb{R}^{d_l}$ (for children only)

Total feature vector:

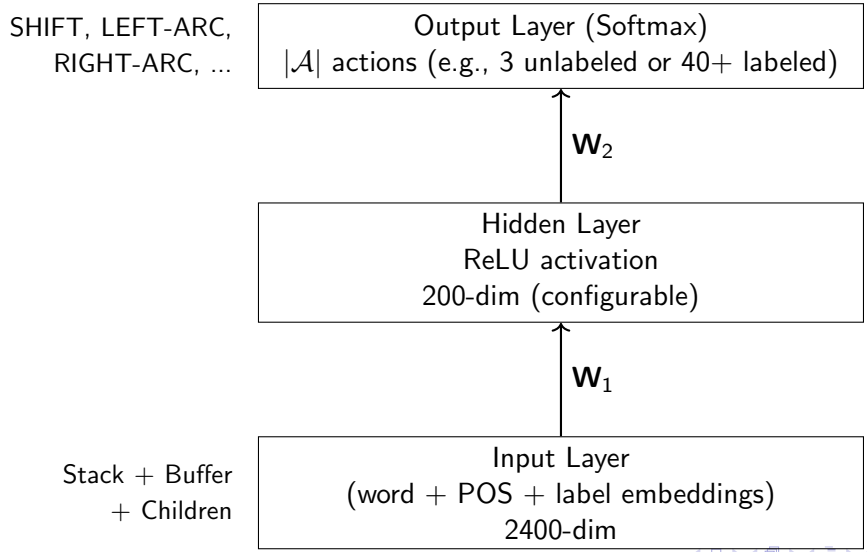
- 18 word embeddings: $18 \times d_w$
- 18 POS tag embeddings: $18 \times d_t$
- 12 label embeddings (only for children): $12 \times d_l$

Example dimensions (Chen & Manning):

- $d_w = 50$, $d_t = 50$, $d_l = 50$
- Total: $18 \times 50 + 18 \times 50 + 12 \times 50 = 2400$ dimensions

Compare to: 10 million sparse features in traditional parser!

Neural Network Architecture



Mathematical Details

Input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$:
Concatenation of all embeddings:

$$\mathbf{x} = [\mathbf{e}^w_{s_1}; \mathbf{e}^w_{s_2}; \ldots; \mathbf{e}^t_{s_1}; \mathbf{e}^t_{s_2}; \ldots; \mathbf{e}^l_{lc_1(s_1)}; \ldots]$$

Hidden layer:

$$\mathbf{h} = \max(0, \mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d_h \times d_{\text{in}}}$, $d_h = 200$

Output layer (action scores):

$$\mathbf{p} = \text{softmax}(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$

where $\mathbf{W}_2 \in \mathbb{R}^{|\mathcal{A}| \times d_h}$

Prediction: $a^* = \arg\max_a p_a$ (among valid actions)

Training: Oracle and Loss

Training data: Treebank with gold dependency trees
Oracle:

- For each configuration during parsing
- Determine gold action that leads toward gold tree
- Dynamic oracle: choose action that keeps gold tree reachable

Loss function: Cross-entropy

$$\mathcal{L} = -\sum_{i=1}^{N} \log p(a_i^* \mid c_i)$$

where a_i^* is gold action for configuration c_i
Optimization:

- AdaGrad with mini-batches
- Learning rate: 0.01 (initial)
- Dropout on hidden layer (50%)
- Pre-train word embeddings or random initialization

Initialization Strategies

Word embeddings:

- **Option 1**: Random initialization (Gaussian)
- **Option 2**: Pre-trained (word2vec, GloVe) — Better!
- Fine-tune during training

POS tag embeddings:

- Random initialization (small vocabulary, 45 tags)
- Learn from scratch

Label embeddings:

- Random initialization (40 relations)
- Learn from scratch

Special tokens:

- NULL: for non-existent children
- ROOT: special token
- UNK: unknown words (rare in training)

Running Example: "I eat fish"

Let's trace through the neural parser

We'll see:

1. Configuration at each step
2. Feature extraction (which embeddings)
3. Neural network input
4. Action prediction

Assume pre-trained embeddings:

- $\mathbf{e}^w_{\text{I}} \in \mathbb{R}^{50}$ (word embedding for "I")
- $\mathbf{e}^t_{\text{PRON}} \in \mathbb{R}^{50}$ (POS embedding for pronoun)
- etc.

For clarity, we'll focus on which embeddings are extracted, not their exact values

Initial Configuration

Configuration:

- Stack: [ROOT]
- Buffer: [I, eat, fish]
- Arcs: $\emptyset$

Feature extraction:

Element	Word	POS
s_1	ROOT	ROOT
s_2	NULL	NULL
s_3	NULL	NULL
b_1	I	PRON
b_2	eat	VERB
b_3	fish	NOUN
$lc_1(s_1), rc_1(s_1), \ldots$	NULL	NULL

Input vector: Concatenate all embeddings (2400-dim)

Network predicts: SHIFT (move "I" to stack)

After First SHIFT

Configuration:

- Stack: [ROOT, I]
- Buffer: [eat, fish]
- Arcs: $\emptyset$

Feature extraction:

Element	Word	POS
s_1	I	PRON
s_2	ROOT	ROOT
s_3	NULL	NULL
b_1	eat	VERB
b_2	fish	NOUN
b_3	NULL	NULL

Network predicts: SHIFT (move "eat" to stack)

Why not LEFT-ARC? Network learns that VERB typically heads PRON, not vice versa

After Second SHIFT

Configuration:

- Stack: [ROOT, I, eat]
- Buffer: [fish]
- Arcs: $\emptyset$

Feature extraction:

Element	Word	POS
s_1	eat	VERB
s_2	I	PRON
s_3	ROOT	ROOT
b_1	fish	NOUN
b_2	NULL	NULL
b_3	NULL	NULL

Network sees:

- VERB on top, PRON second
- NOUN in buffer

After LEFT-ARC

Configuration:

- Stack: [ROOT, eat]
- Buffer: [fish]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}\}$

Feature extraction:

Element	Word	POS	Label
s_1	eat	VERB	—
s_2	ROOT	ROOT	—
$lc_1(s_1)$	I	PRON	nsubj
$rc_1(s_1)$	NULL	NULL	NULL

Key change: Now $lc_1(s_1)$ exists! "I" is leftmost child of "eat"

Network sees:

- VERB with a subject already attached
- NOUN in buffer (likely object)

Network predicts: SHIFT

After SHIFT (fish)

Configuration:

- Stack: [ROOT, eat, fish]
- Buffer: [ ]
- Arcs: $\{\text{eat} \xrightarrow{\text{nsubj}} \text{I}\}$

Feature extraction:

Element	Word	POS	Label
s_1	fish	NOUN	—
s_2	eat	VERB	—
b_1	NULL	NULL	—
$lc_1(s_2)$	I	PRON	nsubj

Network sees:

- NOUN on top, VERB second
- VERB already has subject
- Buffer empty

Network predicts: RIGHT-ARC_{obj} (eat $\rightarrow$ fish)

Neural Network Decision Process

At each step:

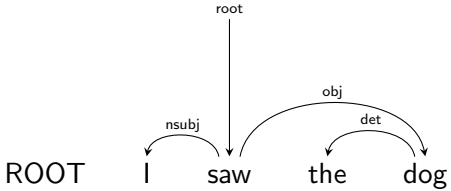
1. **Extract features**:
   - Lookup embeddings for 18 elements
   - Concatenate into input vector $\mathbf{x}$ (2400-dim)
2. **Forward pass**:
   - Hidden: $\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ (200-dim)
   - Scores: $\mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$ (action-dim)
   - Probabilities: $\mathbf{p} = \text{softmax}(\mathbf{s})$
3. **Select action**:
   - Filter invalid actions (preconditions)
   - Choose $a^* = \arg\max_{a \text{ valid}} p_a$

Key insight: Network learns to recognize patterns

- VERB + PRON $\rightarrow$ likely nsubj relation
- VERB (with nsubj) + NOUN $\rightarrow$ likely obj relation

Longer Example: "I saw the dog"

Target dependencies:



Action sequence:

Step	Action	Stack	Buffer
0	—	[ROOT]	[I, saw, the, dog]
1	SHIFT	[ROOT, I]	[saw, the, dog]
2	SHIFT	[ROOT, I, saw]	[the, dog]
3	LEFT-ARC$_{\text{nsubj}}$	[ROOT, saw]	[the, dog]
4	SHIFT	[ROOT, saw, the]	[dog]
5	SHIFT	[ROOT, saw, the, dog]	[ ]
6	LEFT-ARC$_{\text{det}}$	[ROOT, saw, dog]	[ ]
7	RIGHT-ARC$_{\text{obj}}$	[ROOT, saw]	[ ]
8	RIGHT-ARC$_{\text{root}}$	[ROOT]	[ ]

Step 4-5: Processing "the dog"

After step 3:

- Stack: [ROOT, saw], Buffer: [the, dog]
- Arcs: $\{\text{saw} \rightarrow \text{I}\}$

Step 4 (SHIFT "the"):

- Network sees: VERB "saw" with subject on stack, DET "the" in buffer
- Predicts: SHIFT (determiners don't attach directly to verbs)
- Result: Stack = [ROOT, saw, the]

Step 5 (SHIFT "dog"):

- Network sees: DET "the" on stack, NOUN "dog" in buffer
- Predicts: SHIFT (need to see noun before creating det relation)
- Result: Stack = [ROOT, saw, the, dog]

Network learns to delay attachment until full NP is visible

Step 6: Attaching Determiner

Configuration:

- Stack: [ROOT, saw, the, dog]
- Buffer: [ ]

Feature extraction:

- s_1 = dog (NOUN)
- s_2 = the (DET)
- s_3 = saw (VERB)
- b_1 = NULL
- $lc_1(s_3)$ = I (PRON, nsubj)

Network reasoning:

- NOUN on top, DET second $\rightarrow$ DET modifies NOUN
- Predicts: LEFT-ARC$_{\text{det}}$ (dog $\rightarrow$ the)
- Result: Stack = [ROOT, saw, dog]

Compositional Understanding

Why neural networks excel here:

Pattern recognition:

- Hidden layer learns combinations: "VERB with subject + NOUN"
- Automatically discovers that this pattern suggests object relation
- No manual feature engineering needed!

Generalization:

- Embeddings capture similarity: "eat", "saw", "love" are similar verbs
- If "eat fish" $\rightarrow$ obj relation learned, generalizes to "see dog"
- Works for rare word combinations

Context sensitivity:

- Network sees existing children: $lc_1(s_1), rc_1(s_1)$
- Can track: "does this verb already have a subject?"
- Informs decision about current word

Creating Training Data

Input: Treebank with gold dependency trees

Process:

1. For each sentence in training data:
2. Simulate parsing with oracle
3. At each configuration:
   - Extract features $\rightarrow$ create input vector $\mathbf{x}$
   - Get gold action from oracle $\rightarrow$ label y
   - Store $(\mathbf{x}, y)$ as training example

Result:

- Millions of (configuration, action) pairs
- For PTB WSJ: 40,000 sentences $\rightarrow$ 1,000,000 training examples

Batch training:

- Shuffle training examples
- Mini-batches of size 10,000
- Update parameters with AdaGrad

Oracle: Static vs. Dynamic

Static oracle (Chen & Manning):

- Assumes gold path from current configuration
- Simple: always choose action that leads to gold tree
- Problem: If parser makes mistake, no gold action may exist

Dynamic oracle (later work):

- Finds action that keeps gold tree reachable
- More robust to errors during training
- Allows exploration of non-gold paths

Example:

- Gold: eat $\rightarrow$ I (LEFT-ARC), but parser did SHIFT instead
- Static oracle: No gold action available (tree unreachable)
- Dynamic oracle: Find best action given current state

Hyperparameters

Chen & Manning (2014) settings:

Embeddings:

- Word dimension: $d_w = 50$
- POS dimension: $d_t = 50$
- Label dimension: $d_l = 50$
- Pre-trained: word2vec on Wikipedia

Network:

- Hidden layer: 200 units
- Activation: ReLU (later: Cube activation)
- Dropout: 0.5 on hidden layer

Training:

- Optimizer: AdaGrad
- Initial learning rate: 0.01
- Mini-batch size: 10,000
- Epochs: 10-20 until convergence

Cube Activation Function

Surprisingly: Cube activation works better than ReLU!
ReLU:

$$h_i = \max(0, z_i)$$

Cube activation:

$$h_i = z_i^3$$

Why cube?

- Captures higher-order feature interactions
- Allows positive and negative activations (unlike ReLU)
- Empirically gives 0.5% accuracy improvement
- Non-standard but effective for this task

Shows that architecture choices matter for specific tasks!

Performance: Penn Treebank

Dataset: Penn Treebank WSJ (Wall Street Journal)

- Training: Sections 2-21 (40,000 sentences)
- Dev: Section 22
- Test: Section 23

Metric: Unlabeled Attachment Score (UAS)

- % of words with correct head
- Ignores dependency label

Results:

System	UAS
MaltParser (Nivre 2008)	89.8%
Zhang & Nivre (2011)	92.9%
Chen & Manning (2014)	**92.0%**
Chen & Manning + Beam search	92.2%

Speed Comparison

Parsing speed (sentences per second):

System	Speed (sent/sec)
Zhang & Nivre (2011)	90
Chen & Manning (2014)	**654**
Improvement	7.3×

Why so fast?

- **Dense features**: 2400 dimensions vs. 10M sparse features
- **Batch operations**: Matrix multiplications in GPU
- **No feature extraction overhead**: Direct embedding lookup
- **Compact network**: Only 130K parameters (embeddings separate)

Comparable accuracy + 7× faster = Major breakthrough!

Labeled vs. Unlabeled Parsing

Unlabeled parsing:

- Predict: SHIFT, LEFT-ARC, RIGHT-ARC (3 actions)
- Just find tree structure

Labeled parsing:

- Predict: SHIFT, LEFT-ARC$_r$, RIGHT-ARC$_r$ for each relation r
- With 40 relations: $1 + 40 + 40 = 81$ actions
- Find structure + label each arc

Results on PTB:

Metric	**UAS**	**LAS**
Chen & Manning	92.0%	89.7%
Zhang & Nivre	92.9%	91.8%

UAS = unlabeled, LAS = labeled attachment score

Error Analysis

Common error patterns:

1. **PP attachment ambiguity**:
   - "I saw the man with the telescope"
   - Gold: telescope modifies man
   - Pred: telescope modifies saw
   - Classic syntactic ambiguity
2. **Coordination**:
   - "cats and dogs"
   - Complex: first conjunct vs. conjunction vs. second conjunct
   - Parser often gets attachment wrong
3. **Long-distance dependencies**:
   - Words far apart in sentence
   - Greedy parser makes early commitment
   - Beam search helps here

Learned Embeddings: Insights

Word embeddings capture syntactic similarity:

Similar words (nearest neighbors):

- "said": reported, told, announced, stated
- "company": firm, corporation, business, bank
- "big": large, huge, major, small (!)

POS tag embeddings:

- NOUN and PROPN (proper noun) are close
- Different verb forms (VB, VBD, VBZ) cluster together
- Function words (DET, ADP) form their own cluster

Visualization (t-SNE):

- Words group by syntactic role
- Verbs cluster by argument structure
- Learned without explicit syntactic features!

Follow-up Work

Improvements to Chen & Manning:

Weiss et al. (2015): Structured training

- Train on entire sequences, not individual actions
- Beam search during training
- UAS: 93.99% (new state-of-the-art at the time)

Andor et al. (2016): Globally normalized

- Score entire transition sequences
- Beam search with global normalization
- UAS: 94.61%

Dozat & Manning (2017): Biaffine parser

- Graph-based instead of transition-based
- BiLSTM encoder + biaffine attention
- UAS: 95.74% (major jump!)

Modern Parsers

Current state-of-the-art (2024):

BERT-based parsers:

- Encode sentence with pre-trained BERT
- Add biaffine or similar scoring layer
- UAS: 97+ on PTB (nearly saturated)

Multilingual parsing:

- Universal Dependencies: 100+ languages
- Transfer learning from high-resource to low-resource
- Cross-lingual parsers with multilingual BERT

End-to-end models:

- Joint tokenization + POS tagging + parsing
- Character-level input (no explicit tokenization)
- Especially important for morphologically rich languages

Chen & Manning's Lasting Impact

1. **First neural dependency parser**:
   - Showed neural networks competitive with feature-engineered systems
   - Opened flood of neural parsing research
2. **Simplicity**:
   - Single hidden layer feed-forward network
   - Easy to implement and understand
   - Inspired many follow-up works
3. **Speed + accuracy**:
   - 7× faster than previous systems
   - Comparable accuracy
   - Made parsing practical for large-scale applications
4. **Methodology**:
   - Dense representations ¿ sparse features
   - End-to-end learning
   - Set template for neural NLP

Limitations and Remaining Challenges

1. **Greedy decoding**:
   - Cannot recover from early mistakes
   - Beam search helps but adds complexity
2. **Local context**:
   - Only sees top 3 stack, first 3 buffer
   - Misses longer-range dependencies
   - Modern parsers use full sentence encoders
3. **Fixed feature set**:
   - Still relies on specific elements ($s_1, s_2, b_1, \ldots$)
   - Not as flexible as attention mechanisms
4. **Projectivity**:
   - Arc-standard assumes projective trees
   - Extensions needed for non-projective parsing

Key Points

Dependency parsing:

- Represents syntax as word-to-word relations
- Tree structure with head-dependent arcs
- Useful for downstream NLP tasks

Transition-based parsing:

- Build tree incrementally with SHIFT, LEFT-ARC, RIGHT-ARC
- Stack + buffer configuration
- Greedy: $O(n)$ time for sentence of length n

Chen & Manning neural parser:

- Replace sparse features with dense embeddings
- Simple feed-forward network (1 hidden layer)
- $7\times$ faster than previous systems
- Comparable accuracy without manual features

Conceptual Contributions

Before Chen & Manning:

- Millions of hand-crafted sparse features
- Feature engineering is key bottleneck
- Slow feature extraction

After Chen & Manning:

- **Learned representations**: Word/POS/label embeddings
- **Automatic feature learning**: Hidden layer discovers patterns
- **Fast**: Dense matrix operations
- **Simple**: Easy to implement and understand

Broader impact:

- Showed neural networks work for structured prediction
- Inspired neural approaches across NLP
- Template for modern NLP: embeddings + neural network

Comparison: Traditional vs. Neural

	Traditional	**Neural (C&M)**
Features	10M sparse binary	2400 dense continuous
Engineering	Manual templates	Learned automatically
Generalization	Poor (rare combos)	Good (embeddings)
Speed	Slow (feature extraction)	Fast (matrix ops)
Accuracy	92.9% UAS	92.0% UAS
Implementation	Complex	Simple

Trade-off: Slight accuracy decrease for massive speed increase + simpler model

Resources

Original paper:

- Chen & Manning (2014): "A Fast and Accurate Dependency Parser using Neural Networks" (EMNLP)

Related work:

- Nivre (2008): "Algorithms for Deterministic Incremental Dependency Parsing"
- Dozat & Manning (2017): "Deep Biaffine Attention for Neural Dependency Parsing"
- Kiperwasser & Goldberg (2016): "Simple and Accurate Dependency Parsing Using Bidirectional LSTM"

Datasets:

- Penn Treebank (English)
- Universal Dependencies (100+ languages)

Software:

- Stanford CoreNLP (includes neural parser)
- spaCy (fast production parser)